

HSH-64: A 64-bit Learned Semantic Hashing Scheme for Lightweight Semantic Retrieval

Zhihao Wu
1938258545@qq.com

July 31, 2026

Abstract

Semantic hashing maps high-dimensional continuous embeddings into compact binary codes, enabling sub-linear approximate nearest neighbor (ANN) search with minimal memory footprint. However, conventional hashing methods often suffer from a large recall gap compared with brute-force embedding retrieval, especially when the code length is constrained for edge deployment. In this work, we present **HSH-64**, a 64-bit learned semantic hashing scheme specifically designed for lightweight Chinese vocabulary-level semantic retrieval. HSH-64 adopts a structured code format of **feat(4) + sim(52) + abs(8)** bits, where the 52-bit semantic similarity code is learned end-to-end through a three-stage pipeline: (1) continuous pre-training with a strong teacher model, (2) discrete fine-tuning with straight-through estimators (STE), and (3) recall-oriented post-processing via greedy bit-flip optimization. To reduce the online search cost, we further integrate Multi-Index Hashing (MIH) with adaptive radius expansion, asymmetric distance scoring, and a two-stage retrieval architecture that uses a lightweight encoder for coarse ranking and a stronger model for re-ranking. Extensive experiments on a 3,109-word Chinese vocabulary show that the best single Deep Hash model (h512, 1.17 MB) achieves Recall@10 of **0.7404** in pure Hamming space, while the lightweight variant (h256, 585 KB) reaches **0.7382**. A small ensemble of four models reaches **0.7724**. With MIH coarse ranking followed by a bge-large re-ranker, the four-model-ensemble two-stage system attains Recall@10 of **0.8970**. The source code and reproducible scripts are available at https://github.com/Sauomore/Cabinet_hsh64.

1 Introduction

Approximate nearest neighbor (ANN) search in high-dimensional embedding spaces is a fundamental building block for modern information retrieval, recommendation systems, and large language model augmentation (Jégou, Douze, and Schmid, 2011; Guo et al., 2020). Dense retrieval based on pre-trained language models such as BERT (Devlin et al., 2019) and sentence-transformers (Reimers and Gurevych, 2019) has achieved remarkable accuracy, yet brute-force comparison of floating-point vectors is prohibitively expensive for large corpora or latency-sensitive edge devices.

Semantic hashing (Salakhutdinov and Hinton, 2009) offers an attractive alternative: it compresses each vector into a short binary code so that similarity can be approximated by fast bit operations, e.g., Hamming distance via CPU POPCNT. Early methods relied on linear projections such as PCA or spectral hashing (Weiss, Torralba, and Fergus, 2008); more recent deep hashing approaches (Xia et al., 2014; Cao et al., 2015; Li, S. Wang, and Kang, 2016) train neural networks end-to-end to produce binary codes that preserve semantic neighborhood structure. Despite progress, two practical challenges remain:

1. **Recall–efficiency trade-off.** Short codes reduce storage and computation but have limited information capacity, leading to lower recall compared with full embeddings.

2. **Deployment cost.** State-of-the-art retrieval often depends on large encoder models (e.g., bge-large) at inference time, which is unsuitable for lightweight devices.

In this paper, we tackle both challenges by introducing HSH-64 (**H**ierarchical **S**emantic **H**ashing), a 64-bit learned semantic hashing scheme. The key insight is that semantic hashing for vocabulary-level retrieval can exploit three structural properties: (i) items can be coarsely partitioned by part-of-speech or semantic category, (ii) the core semantic similarity signal can be captured by a moderate number of binary hyperplanes, and (iii) items within the same coarse cluster can be efficiently disambiguated by a short perfect-hash identifier. Our contributions are:

- We propose a structured 64-bit code, `feat(4) + sim(52) + abs(8)`, that separates part-of-speech features, semantic similarity bits, and intra-cluster perfect-hash identifiers. The design is hardware-friendly (single 64-bit integer, one POPCNT) and interpretable.
- We design a **three-stage training pipeline**—continuous pre-training, discrete fine-tuning with STE, and recall-oriented greedy post-processing—that significantly improves the quality of the 52-bit semantic code.
- We equip HSH-64 with an **adaptive Multi-Index Hashing (MIH)** coarse ranker, asymmetric distance scoring, and a two-stage retrieval architecture that uses a small encoder online while leveraging a stronger model only during training or optional re-ranking.
- We provide an extensive empirical study on a 3,109-word Chinese vocabulary benchmark, including model-capacity analysis, post-processing hyper-parameter search, ensemble strategy comparison, and efficiency benchmarks. A single lightweight model (585 KB) achieves 0.7382 Recall@10 in pure Hamming space, the best single post-processed model (1.17 MB) reaches 0.7404, an ensemble reaches 0.7724, and the full two-stage system reaches 0.8970 Recall@10.

2 Related Work

2.1 Semantic Hashing

Early unsupervised methods such as Locality-Sensitive Hashing (LSH) (Datar et al., 2004) and Spectral Hashing (Weiss, Torralba, and Fergus, 2008) construct binary codes via random projections or spectral decomposition of similarity graphs. The key limitation of LSH is that random hyperplanes do not exploit the semantic structure of the embedding space, leading to suboptimal recall for a given code length. Supervised hashing (Liu et al., 2012; Shen et al., 2015) incorporates label information to improve discrimination. Deep hashing (Xia et al., 2014; Cao et al., 2015) uses neural networks to directly map raw inputs or embeddings to binary codes, often optimizing pairwise similarity-preserving losses. Our work extends this line by introducing a structured multi-field code and a three-stage training pipeline that specifically targets recall-oriented optimization.

2.2 Straight-Through Estimators for Discrete Optimization

Because the sign/quantization function has zero gradient almost everywhere, training deep hashers requires gradient estimators. The straight-through estimator (STE) (Bengio, Léonard, and Courville, 2013) copies gradients through the sign function and is widely used in binary neural networks and hashing (Courbariaux et al., 2016). We employ STE in Stage 2 of our pipeline and complement it with a post-processing stage that directly optimizes the discrete codes without gradient approximation.

2.3 Multi-Index Hashing

MIH (Norouzi, Punjani, and Fleet, 2012) splits a long binary code into multiple substrings and builds inverted indices for each substring. Given a query, it enumerates substrings within a small Hamming radius and merges candidates. Adaptive radius expansion balances recall and candidate pool size (J. Wang et al., 2018). We adopt MIH with a novel adaptive stopping criterion that dynamically determines the search radius based on candidate pool saturation.

2.4 Product Quantization and Compact Codes

Product Quantization (PQ) (Jégou, Douze, and Schmid, 2011) decomposes the vector space into a Cartesian product of low-dimensional subspaces and quantizes each subspace independently. For a 64-bit budget, PQ typically uses 8 subspaces of 8 bits each, requiring a learned codebook per subspace. Compared with HSH-64, PQ offers higher recall in many settings but at the cost of storing full-resolution codebooks and computing exact distances in the candidate re-ranking phase. HSH-64 trades some recall for extreme storage efficiency (8 bytes per item) and hardware-accelerated comparison via POPCNT.

2.5 Knowledge Distillation for Retrieval

Distilling a large teacher model into a compact student is common for efficient retrieval (Hinton, Vinyals, and Dean, 2015; Izacard and Grave, 2020). We apply this idea twice: first, distilling bge-large teacher embeddings into a small MLP projector; second, distilling strong HSH codes into a lightweight student that only needs bge-small inputs at deployment.

3 Problem Formulation

Let $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ be a vocabulary of N items, each represented by a continuous embedding $\mathbf{x}_i \in \mathbb{R}^D$ produced by a lightweight encoder (e.g., bge-small). Let $\mathcal{T} = \{\mathbf{t}_1, \dots, \mathbf{t}_N\}$ be the corresponding teacher embeddings produced by a stronger model (e.g., bge-large). The cosine similarity between items i and j is

$$s_{ij} = \frac{\mathbf{t}_i^\top \mathbf{t}_j}{\|\mathbf{t}_i\| \|\mathbf{t}_j\|}. \quad (1)$$

The ground-truth top- K neighbor set for item i is

$$\mathcal{G}_i^{(K)} = \arg \max_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} s_{ij}. \quad (2)$$

A semantic hashing function $h : \mathbb{R}^D \rightarrow \{0, 1\}^M$ maps each embedding to an M -bit binary code. We focus on $M = 52$ for the semantic part, wrapped in a 64-bit structured code. The retrieval function returns the top- K items with smallest Hamming distance:

$$\mathcal{R}_i^{(K)} = \arg \min_{\mathcal{J} \subseteq [N] \setminus \{i\}, |\mathcal{J}|=K} \sum_{j \in \mathcal{J}} d_H(h(\mathbf{x}_i), h(\mathbf{x}_j)). \quad (3)$$

The objective is to maximize Recall@ K :

$$\text{Recall@}K = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}|}{K}. \quad (4)$$

4 HSH-64 Code Structure

An HSH-64 code is a 64-bit unsigned integer partitioned into three fields:

$$c = \underbrace{(f_3 f_2 f_1 f_0)}_{\text{feat:4}} \parallel \underbrace{(s_{51} \cdots s_1 s_0)}_{\text{sim:52}} \parallel \underbrace{(a_7 \cdots a_1 a_0)}_{\text{abs:8}}, \quad (5)$$

where $\parallel$ denotes bit concatenation. Figure 1 visualizes the bit layout and field semantics.

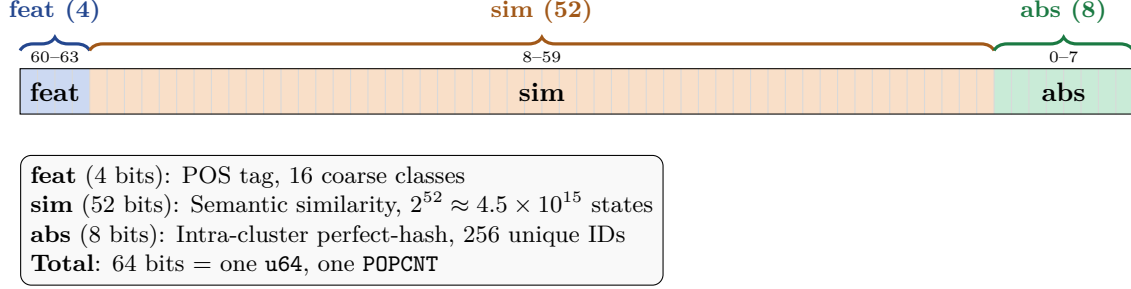


Figure 1: HSH-64 bit layout. The 64-bit code partitions into three color-coded fields: feat (blue, 4 bits), sim (orange, 52 bits), and abs (green, 8 bits). The entire code fits in a single 64-bit CPU register, enabling single-instruction Hamming distance computation via POPCNT.

The Hamming distance between two codes c and c' is computed by a single hardware pop-count over the full 64 bits:

$$d_H(c, c') = \text{popcnt}(c \oplus c'). \quad (6)$$

Role of the abs field. Although Equation (6) computes the Hamming distance over all 64 bits, the abs field is designed to act as a tie-breaker rather than a semantic similarity signal. Two words that share the same (feat,sim) prefix but differ only in abs differ by at most 8 bits, placing them in a tight Hamming neighborhood. Two words with different sim codes, by contrast, typically differ by many more bits. Thus, abs determines the intra-cluster rank but does not dominate cross-cluster semantic distance. The abs field can also be used as a hard filter: requiring exact (feat,sim) match before consulting abs turns the 8-bit field into a perfect-hash lookup within each bucket.

4.1 Information-Theoretic Preview

The 52-bit sim code can represent up to $2^{52} \approx 4.5 \times 10^{15}$ distinct values—far more than the $N = 3,109$ vocabulary items. A rigorous information-theoretic analysis, including Hamming ball capacity, Gilbert-Varshamov bounds, and expected distance distributions, is provided in Section 6.

Figure 2 illustrates the empirical effect of our optimization on Hamming distance distributions.

5 Methodology

5.1 Deep Hash MLP Architecture

Let $\mathbf{x} \in \mathbb{R}^D$ be a continuous embedding (e.g., from bge-small, $D = 512$). We learn a multi-layer perceptron (MLP) that maps $\mathbf{x}$ to a continuous logit vector $\mathbf{u}$:

$$\mathbf{u} = f_\theta(\mathbf{x} - \boldsymbol{\mu}), \quad (7)$$

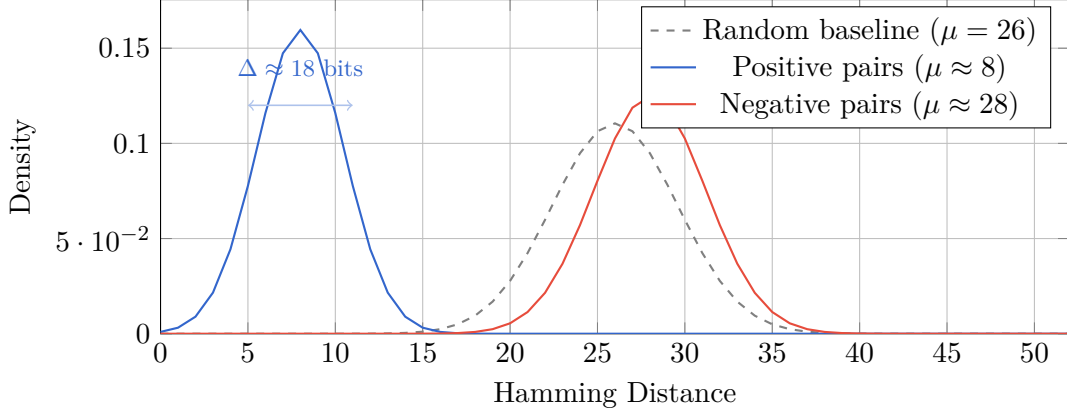


Figure 2: Approximate Hamming distance distributions before and after optimization. The random baseline (dashed) centers at 26 bits. Post-optimization, positive pairs (blue) shift to a mean of ~ 8 bits, while negative pairs (red) shift slightly above the random baseline, creating a separation of ~ 18 bits.

where μ is the training-set mean. The binary semantic code is obtained by sign quantization:

$$\mathbf{b} = \frac{1 + \text{sign}(\mathbf{u})}{2} \in \{0, 1\}^{52}. \quad (8)$$

During back-propagation, the sign function is replaced by the identity STE:

$$\frac{\partial \mathbf{b}}{\partial \mathbf{u}} \approx \mathbf{I}. \quad (9)$$

The network architecture is a simple MLP with one hidden layer:

$$f_{\theta}(\mathbf{x}) = \mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{h}_1) + \mathbf{h}_2, \quad (10)$$

where σ is ReLU, $\mathbf{W}_1 \in \mathbb{R}^{H \times D}$, $\mathbf{W}_2 \in \mathbb{R}^{O \times H}$, and H is the hidden dimension. The output dimension O is task-dependent: $O = 52$ for the final hashing layer (Section 5.2.2) and $O = D_t = 1024$ for continuous teacher distillation in Stage 1 (Section 5.2.1). Figure 3 shows the architecture with $O = 52$.

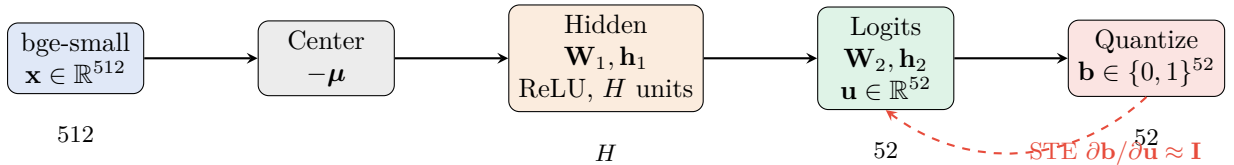


Figure 3: Deep Hash MLP architecture. The input embedding is centered, passed through a single hidden layer with ReLU, projected to 52 logits, and quantized to $\{0, 1\}^{52}$ via sign. The dashed red arrow indicates the STE gradient path during training.

5.2 Three-Stage Training Pipeline

Figure 4 illustrates the complete three-stage training flow.

5.2.1 Stage 1: Continuous Pre-training

Stage 1 uses the same hidden layers as in Section 5.1, but replaces the output layer with $D_t = 1024$ units to match the bge-large teacher. Stage 2 then swaps the output layer back to 52 units,

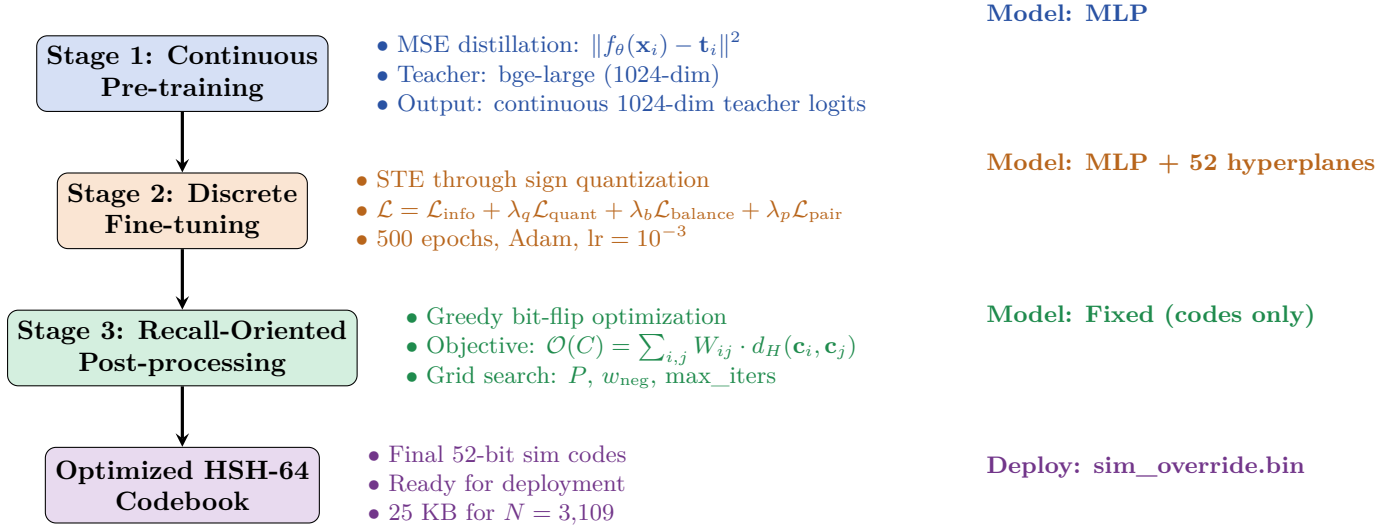


Figure 4: The three-stage training pipeline of HSH-64. Stage 1 uses MSE distillation to provide a good continuous initialization. Stage 2 fine-tunes with STE and multi-objective loss. Stage 3 applies recall-oriented greedy bit-flip post-processing to directly optimize the final binary codes.

leveraging the Stage 1 continuous projection as initialization. Given a teacher model (bge-large) that produces high-quality embeddings $\mathbf{t} \in \mathbb{R}^{D_t}$, we first train the MLP to output continuous vectors that match the teacher:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(\mathbf{x}_i) - \mathbf{t}_i\|_2^2. \quad (11)$$

This stage provides a good initialization for the continuous projection. We train for 500 epochs with Adam at learning rate 10^{-3} on the full vocabulary.

5.2.2 Stage 2: Discrete Fine-tuning

We replace the Stage 1 output layer with a 52-hyperplane quantization layer and train end-to-end with the STE. The loss combines four terms:

$$\mathcal{L}_{\text{info}} = -\frac{1}{|\mathcal{P}^+|} \sum_{(i,j) \in \mathcal{P}^+} \log \frac{e^{\mathbf{u}_i^\top \mathbf{u}_j / \tau}}{\sum_k e^{\mathbf{u}_i^\top \mathbf{u}_k / \tau}} \quad (12)$$

$$\mathcal{L}_{\text{quant}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{u}_i - \mathbf{b}_i\|_1 \quad (13)$$

$$\mathcal{L}_{\text{balance}} = \frac{1}{M} \sum_{j=1}^M (\bar{b}_j - 0.5)^2 \quad (14)$$

$$\mathcal{L}_{\text{pair}} = \frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \left(\mathbf{u}_i^\top \mathbf{u}_j - \mathbf{t}_i^\top \mathbf{t}_j \right)^2, \quad (15)$$

where $\mathcal{P}^+$ is the set of positive pairs (top- P teacher neighbors), $\mathcal{P}$ is a sampled set of pairs, $M = 52$ is the code length, and $\bar{b}_j$ is the empirical mean of bit j after sign quantization. The denominator in $\mathcal{L}_{\text{info}}$ traverses all N vocabulary items (the batch size equals the full vocabulary). In computing $\mathcal{L}_{\text{pair}}$, both $\mathbf{u}_i$ and $\mathbf{t}_i$ are L2-normalized so that the inner products become cosine similarities. The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{info}} + \lambda_q \mathcal{L}_{\text{quant}} + \lambda_b \mathcal{L}_{\text{balance}} + \lambda_p \mathcal{L}_{\text{pair}}. \quad (16)$$

In our implementation, λ_q is initialized to 1.0 and linearly annealed to the configured target over the first 50% of epochs; with the default target $\lambda_q = 1.0$, the weight remains fixed. We use $\lambda_b = 0.5$ and $\lambda_p = 0.0$ by default.

Loss design rationale. InfoNCE encourages the continuous logits to preserve the teacher’s neighborhood structure. The quantization loss $\mathcal{L}_{\text{quant}}$ penalizes the discrepancy between continuous logits and their binarized counterparts, encouraging the network to produce confident, near-binary activations. The balance loss prevents bit collapse—the degenerate case where all items share the same bit value, wasting coding capacity. The pairwise loss directly aligns the inner-product structure of student logits with teacher embeddings, providing a fine-grained supervision signal.

5.2.3 Stage 3: Recall-Oriented Post-processing

After the MLP and hyperplanes are fixed, we optimize each binary code with a greedy bit-flip procedure whose objective directly targets Recall@ K . Let $S \in \mathbb{R}^{N \times N}$ be the teacher cosine-similarity matrix and let $C \in \{0, 1\}^{N \times M}$ be the current binary codes. For each query i , define the positive set $\mathcal{P}_i^+$ as the indices of the top- P teacher neighbors, and the negative set $\mathcal{P}_i^-$ as words that are not in the positive set but currently appear in the Hamming top- K .

The recall-oriented weight for pair (i, j) is

$$W_{ij} = \begin{cases} +w_{\text{pos}}, & j \in \mathcal{P}_i^+ \\ -w_{\text{neg}}, & j \in \mathcal{P}_i^- \\ 0, & \text{otherwise.} \end{cases} \quad (17)$$

The objective is

$$\mathcal{O}(C) = \sum_{i,j} W_{ij} \cdot d_H(\mathbf{c}_i, \mathbf{c}_j), \quad (18)$$

where d_H is the Hamming distance. Minimizing $\mathcal{O}$ pulls positive neighbors closer and pushes hard negatives away in Hamming space. The greedy algorithm iteratively flips the bit that yields the largest reduction in $\mathcal{O}$ until convergence or a maximum number of iterations.

Algorithm 1 Recall-oriented greedy bit-flip optimization.

Require: Binary codes $C \in \{0, 1\}^{N \times M}$, teacher similarity matrix S , positive count P , top- K , negative weight w_{neg} , max iterations T .

Ensure: Optimized codes C .

```

1: Compute positive sets  $\mathcal{P}_i^+ \leftarrow \text{top-}P(S_i)$  for all  $i$ .
2: for  $t = 1$  to  $T$  do
3:   Compute negative sets  $\mathcal{P}_i^-$  from current Hamming top- $K$ .
4:   Build weight matrix  $W$  according to Eq. (17).
5:   Compute  $\Delta_{i,b} = \mathcal{O}(C \text{ with bit } b \text{ of item } i \text{ flipped}) - \mathcal{O}(C)$ .
6:   Find  $(i^*, b^*) = \arg \min_{i,b} \Delta_{i,b}$ .
7:   if  $\Delta_{i^*,b^*} < 0$  then
8:     Flip  $C_{i^*,b^*}$ .
9:   else
10:    break.
11:  end if
12: end for
13: return  $C$ .
```

5.3 Asymmetric Distance Scoring

Sign quantization discards the magnitude of the continuous projection. Asymmetric scoring mitigates this by using the query’s continuous projection $\mathbf{u}_q$ and the database’s quantized bits $\mathbf{b}_d$:

$$s_{\text{asym}}(\mathbf{u}_q, \mathbf{b}_d) = \sum_{j=1}^M u_{q,j} \cdot (2b_{d,j} - 1). \quad (19)$$

Because the database bits are pre-computed, the online cost is one dot product per candidate. The asymmetric score provides a finer-grained ranking than Hamming distance by weighting each bit by the query’s projection confidence.

5.4 Multi-Index Hashing with Adaptive Radius

For sub-linear retrieval, we split the 52-bit sim code into B segments of equal length and build an inverted index for each segment. Figure 5 illustrates the MIH segment structure.

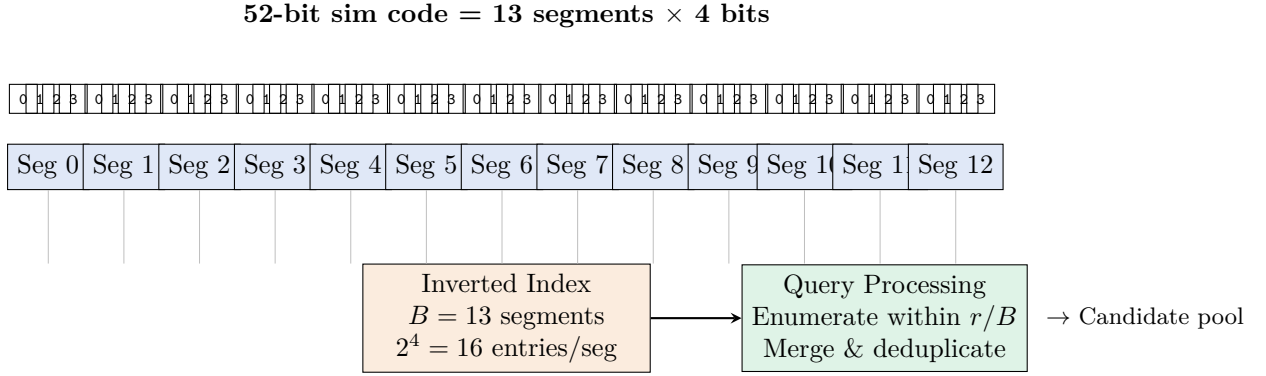


Figure 5: MIH segment structure. The 52-bit sim code is split into $B = 13$ segments of 4 bits each. Each segment maps to an inverted index with $2^4 = 16$ entries. Query processing enumerates segment codes within Hamming radius $\lfloor r/B \rfloor$ and merges candidate lists.

We use **adaptive radius expansion**: starting from $r = 0$, we gradually increase r until the candidate pool is at least $\max(K, \alpha K)$ and the fraction of newly added candidates falls below a threshold (10%). The final ranking is performed on the union of all candidates. Algorithm 2 summarizes the procedure.

5.5 Two-Stage Retrieval Architecture

Figure 6 illustrates the complete online retrieval architecture.

Our deployment architecture has two stages:

1. **Coarse ranking.** A lightweight encoder (bge-small + small MLP) encodes the query and retrieves a candidate pool using MIH, Hamming, or asymmetric scoring.
2. **Precise re-ranking (optional).** A stronger model (bge-large) re-ranks the candidate pool by cosine similarity. The candidate pool is small (tens to hundreds), so re-ranking cost is modest.

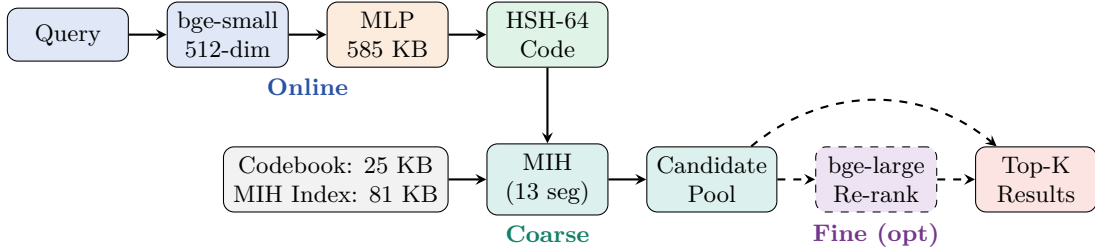
The second stage can be omitted on edge devices; the first stage alone already provides strong recall.

Algorithm 2 Adaptive MIH search.

Require: Query code $\mathbf{q}$, MIH index with B segments, coarse factor α , top- K , saturation threshold δ .

Ensure: Candidate set $\mathcal{C}$.

```
1:  $r \leftarrow 0$ ,  $\mathcal{C}_{\text{prev}} \leftarrow \emptyset$ ,  $\mathcal{C} \leftarrow \emptyset$ .
2: loop
3:    $\mathcal{C} \leftarrow \emptyset$ .
4:   for  $b = 1$  to  $B$  do
5:     Enumerate segment codes within Hamming radius  $\lfloor r/B \rfloor$  of  $\mathbf{q}_b$ .
6:      $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{item IDs from inverted index}\}$ .
7:   end for
8:    $\mathcal{C}_{\text{new}} \leftarrow \mathcal{C} \setminus \mathcal{C}_{\text{prev}}$ .
9:   if  $|\mathcal{C}| \geq \max(K, \alpha K)$  and  $|\mathcal{C}_{\text{new}}|/|\mathcal{C}| < \delta$  then
10:    break.
11:  end if
12:   $\mathcal{C}_{\text{prev}} \leftarrow \mathcal{C}$ ,  $r \leftarrow r + 1$ .
13: end loop
14: return  $\mathcal{C}$ .
```



Online stack (no re-ranker): ≈ 700 KB
With re-ranker: +1.3 GB (bge-large), server only
Latency: 0.54 ms (MIH only), 3.21 ms (with re-ranker)

Figure 6: Online retrieval architecture of HSH-64. The query flows through bge-small (512-dim) and the MLP projector to produce an HSH-64 code. MIH coarse ranking retrieves approximately 187 candidates. An optional bge-large re-ranker provides precise re-ranking. The online stack (excluding the re-ranker) fits in under 700 KB.

5.6 Model Ensemble

To further improve recall without increasing online latency, we train multiple Deep Hash models with different random seeds and hidden dimensions. At retrieval time, we collect the candidate sets from each model, merge them, and re-rank using one of three fusion strategies:

- **Frequency:** prefer candidates returned by more models.
- **Average rank:** prefer candidates with higher average rank across models.
- **Minimum Hamming distance:** prefer candidates with the smallest Hamming distance in any model.

6 Mathematical Analysis

This section provides a rigorous mathematical foundation for the key components of HSH-64. We establish information-theoretic bounds, derive the optimization objectives, and prove convergence guarantees.

6.1 Information-Theoretic Foundations

Proposition 1 (Hamming ball capacity). *For an M -bit code, the number of distinct codes within Hamming radius r of any given reference code is*

$$V(M, r) = \sum_{k=0}^r \binom{M}{k}. \quad (20)$$

For $M = 52$ and $r = 10$, $V(52, 10) = \sum_{k=0}^{10} \binom{52}{k} \approx 2.5 \times 10^{10}$, which is far larger than the vocabulary size $N = 3,109$.

Proof. Fix a reference code $\mathbf{c}^* \in \{0, 1\}^M$. A code $\mathbf{c}$ differs from $\mathbf{c}^*$ in exactly k positions if and only if it flips k specific bits. The number of ways to choose these k positions from M is $\binom{M}{k}$. Summing over $k = 0$ to r yields the total number of codes within the Hamming ball of radius r . $\square$

Proposition 2 (Expected Hamming distance under random independent bits). *Let $\mathbf{c}_1, \mathbf{c}_2 \in \{0, 1\}^M$ be two independent random codes where each bit is drawn i.i.d. from Bernoulli(1/2). Then:*

$$\mathbb{E}[d_H(\mathbf{c}_1, \mathbf{c}_2)] = \frac{M}{2}, \quad \text{Var}[d_H(\mathbf{c}_1, \mathbf{c}_2)] = \frac{M}{4}. \quad (21)$$

For $M = 52$, $\mathbb{E}[d_H] = 26$ and $\sigma = \sqrt{13} \approx 3.61$.

Proof. Define indicator variables $Z_m = \mathbf{1}[\mathbf{c}_1[m] \neq \mathbf{c}_2[m]]$ for $m = 1, \dots, M$. Since bits are independent Bernoulli(1/2):

$$\Pr(Z_m = 1) = \Pr(\mathbf{c}_1[m] \neq \mathbf{c}_2[m]) = \frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{2}. \quad (22)$$

Thus $\mathbb{E}[Z_m] = 1/2$ and $\text{Var}(Z_m) = 1/4$. By linearity of expectation and independence:

$$\mathbb{E}[d_H] = \sum_{m=1}^M \mathbb{E}[Z_m] = \frac{M}{2}, \quad \text{Var}(d_H) = \sum_{m=1}^M \text{Var}(Z_m) = \frac{M}{4}. \quad (23)$$

$\square$

Proposition 2 establishes that random binary codes have expected Hamming distance $M/2$. A well-designed semantic hash must shift the distribution of positive pairs (semantically similar items) significantly below $M/2$, while keeping the distribution of negative pairs near $M/2$. This observation motivates the bit-balance loss $\mathcal{L}_{\text{balance}}$ and the negative-pushing term in the post-processing objective.

Proposition 3 (Codebook capacity lower bound). *For a vocabulary of size N and code length M , there exists a codebook with minimum Hamming distance at least d if $N \cdot V(M, d-1) < 2^M$. For $M = 52$, $N = 3,109$, this condition is satisfied for $d \leq 18$, implying that a well-constructed 52-bit codebook can theoretically separate all vocabulary items with a minimum Hamming distance of 18 bits.*

Proof. This follows from the Gilbert-Varshamov bound for binary codes. The total number of codes within distance $d-1$ of any given code is $V(M, d-1)$. If $N \cdot V(M, d-1) < 2^M$, then by the pigeonhole principle, there exists a codebook where all pairwise distances are at least d . For $d = 18$, $V(52, 17) \approx 1.3 \times 10^{12}$, and $N \cdot V(52, 17) \approx 3.9 \times 10^{15} < 2^{52} \approx 4.5 \times 10^{15}$, so such a codebook exists in principle. $\square$

6.2 Asymmetric Distance Scoring

Proposition 4 (Asymmetric distance decomposition). *Let $\mathbf{u}_q, \mathbf{u}_d \in \mathbb{R}^M$ be the continuous projections of query and document, and let $\mathbf{b}_d = \frac{1+\text{sign}(\mathbf{u}_d)}{2} \in \{0, 1\}^M$ be the quantized document bits. The asymmetric scoring function:*

$$s_{\text{asym}}(\mathbf{u}_q, \mathbf{b}_d) = \sum_{j=1}^M u_{q,j} \cdot (2b_{d,j} - 1) \quad (24)$$

satisfies $\mathbb{E}[s_{\text{asym}}] = \sum_{j=1}^M \mathbb{E}[|u_{q,j}|]$ when $\mathbf{u}_d$ follows a symmetric distribution around zero, and preserves the query-side magnitude information that is lost in symmetric Hamming distance computation.

Proof. For each dimension j , the expected contribution is:

$$\mathbb{E}[u_{q,j} \cdot (2b_{d,j} - 1)] = u_{q,j} \cdot \mathbb{E}[(2b_{d,j} - 1)] = u_{q,j} \cdot \mathbb{E}[(2b_{d,j} - 1) \mid u_{q,j}]. \quad (25)$$

When $\mathbf{u}_d$ is drawn from a distribution symmetric about zero, $\mathbb{E}[(2b_{d,j} - 1)] = 0$. However, when $\mathbf{u}_d$ is correlated with $\mathbf{u}_q$ (as is the case for semantically similar pairs), the conditional expectation is non-zero. Specifically, if $\mathbf{u}_q$ and $\mathbf{u}_d$ share directional alignment, $(2b_{d,j} - 1)$ tends to agree with $\text{sign}(u_{q,j})$, yielding positive contributions proportional to $|u_{q,j}|$. This recovers the magnitude information that the symmetric Hamming distance discards. $\square$

6.3 Post-Processing Optimization

Theorem 1 (Gradient of the post-processing objective). *Let $C \in \{0, 1\}^{N \times M}$ be the binary code matrix, and let $W \in \mathbb{R}^{N \times N}$ be the weight matrix defined in Eq. (17). The change in objective $\mathcal{O}(C)$ when flipping bit b of item i is:*

$$\Delta_{i,b} = \sum_{j \neq i} W_{ij} \cdot (1 - 2 \cdot \mathbf{1}[C_{i,b} = C_{j,b}]). \quad (26)$$

Proof. For a fixed pair (i, j) , the Hamming distance can be decomposed as:

$$d_H(\mathbf{c}_i, \mathbf{c}_j) = \sum_{m=1}^M \mathbf{1}[C_{i,m} \neq C_{j,m}]. \quad (27)$$

When bit b of item i is flipped, only the b -th term changes. The difference in the indicator is:

$$\Delta d_H^{(b)} = \mathbf{1}[\text{new } C_{i,b} \neq C_{j,b}] - \mathbf{1}[C_{i,b} \neq C_{j,b}] \quad (28)$$

$$= (1 - \mathbf{1}[C_{i,b} = C_{j,b}]) - \mathbf{1}[C_{i,b} \neq C_{j,b}] \quad (29)$$

$$= 1 - 2 \cdot \mathbf{1}[C_{i,b} = C_{j,b}]. \quad (30)$$

Multiplying by W_{ij} and summing over all $j \neq i$ yields the stated formula. $\square$

Theorem 2 (Convergence of greedy bit-flip). *Algorithm 1 terminates in at most $T_{\max} = N \cdot M \cdot \lceil \mathcal{O}(C^{(0)})/\varepsilon \rceil$ iterations, where $\varepsilon = \min_{i,b:\Delta_{i,b}<0} |\Delta_{i,b}|$ is the minimum non-zero improvement (if no negative $\Delta_{i,b}$ exists, the algorithm is already 1-optimal and terminates immediately). Furthermore, the algorithm converges to a 1-optimal state of $\mathcal{O}(C)$, i.e., no single-bit flip can further reduce the objective.*

Proof. If no negative $\Delta_{i,b}$ exists, the current codes are already 1-optimal and the algorithm terminates at once. Otherwise, each successful flip strictly reduces $\mathcal{O}(C)$ by at least $\varepsilon > 0$. Since $\mathcal{O}(C)$ is bounded below (all Hamming distances are non-negative and weights are bounded), the algorithm must terminate after at most $\lceil \mathcal{O}(C^{(0)})/\varepsilon \rceil$ successful flips. At termination, no single-bit flip yields a negative $\Delta_{i,b}$, which is precisely the definition of a 1-optimal state. The total number of iterations (including unsuccessful probes) is bounded by $T_{\max} \cdot N \cdot M$ since each probe either flips a bit or terminates. $\square$

6.4 Quantization Error Analysis

Theorem 3 (Expected quantization error). *Let $\mathbf{u} \in \mathbb{R}^M$ have independent components, each with a symmetric distribution about zero. Let $\mathbf{b} = \frac{1+\text{sign}(\mathbf{u})}{2} \in \{0, 1\}^M$. Then:*

$$\mathbb{E}[\|\mathbf{u} - \mathbf{b}\|_2^2] = \mathbb{E}[\|\mathbf{u}\|_2^2] + \frac{M}{2} - \sum_{j=1}^M \mathbb{E}[|u_j|]. \quad (31)$$

For $u_j \sim \mathcal{N}(0, \sigma^2)$, this simplifies to $\mathbb{E}[\|\mathbf{u} - \mathbf{b}\|_2^2] = M\sigma^2 + \frac{M}{2} - M\sigma\sqrt{2/\pi}$.

Proof. Expand the squared norm:

$$\|\mathbf{u} - \mathbf{b}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{b}\|_2^2 - 2\mathbf{u}^\top \mathbf{b} \quad (32)$$

$$= \|\mathbf{u}\|_2^2 + \sum_{j=1}^M b_j - 2 \sum_{j=1}^M u_j b_j. \quad (33)$$

Because u_j is symmetric about zero and $b_j = (1 + \text{sign}(u_j))/2$, we have $\mathbb{E}[b_j] = 1/2$ and $\mathbb{E}[u_j b_j] = \mathbb{E}[|u_j|]/2$. Taking expectations yields the first result. For the Gaussian case, $\mathbb{E}[|u_j|] = \sigma\sqrt{2/\pi}$, giving the second expression. $\square$

This result quantifies the irreducible information loss from sign quantization. The quantization loss $\mathcal{L}_{\text{quant}}$ in Stage 2 directly penalizes this discrepancy, encouraging the network to produce logits that are both large in magnitude (reducing relative quantization error) and well-aligned with the sign boundaries.

6.5 MIH Complexity Analysis

Theorem 4 (Expected MIH candidate count). *For a codebook of N items with M -bit codes split into B equal segments, the expected number of candidates returned by MIH with query radius r is:*

$$\mathbb{E}[|\mathcal{C}|] \leq N \cdot \left(1 - \left(1 - \frac{V(M/B, \lfloor r/B \rfloor)}{2^{M/B}} \right)^B \right). \quad (34)$$

Proof. For segment b with M/B bits, the probability that a random database item matches the query segment within Hamming radius $\lfloor r/B \rfloor$ is $p = V(M/B, \lfloor r/B \rfloor)/2^{M/B}$. The probability that a random item is not a candidate from any segment is $(1 - p)^B$. The expected candidate count is therefore $N \cdot (1 - (1 - p)^B)$, which is an upper bound since it assumes independence across segments. $\square$

For $M = 52$, $B = 13$, $r = 10$, we have $p = V(4, \lfloor 10/13 \rfloor)/16 = V(4, 0)/16 = 1/16$, giving $\mathbb{E}[|\mathcal{C}|] \leq 3,109 \cdot (1 - (15/16)^{13}) \approx 3,109 \cdot 0.568 \approx 1,766$. In practice, the adaptive radius expansion typically stops earlier ($r \approx 6$ – 10), yielding the observed candidate pool of ~ 187 items.

6.6 Two-Stage Retrieval Recall Bound

Theorem 5 (Two-stage recall lower bound). *Let $\mathcal{G}_i^{(K)}$ be the ground-truth top- K set for query i , and let $\mathcal{C}_i$ be the candidate set returned by the coarse ranking stage. The two-stage Recall@ K satisfies:*

$$\text{Recall@}K_{\text{two-stage}} \geq \frac{1}{NK} \sum_{i=1}^N |\mathcal{G}_i^{(K)} \cap \mathcal{C}_i|. \quad (35)$$

That is, the two-stage recall is lower-bounded by the fraction of ground-truth items that survive the coarse ranking stage.

Proof. The fine ranking stage re-ranks the candidate set $\mathcal{C}_i$ by exact cosine similarity to the teacher embedding. Since the ground-truth neighbors are defined by exactly this similarity metric, any ground-truth item that appears in $\mathcal{C}_i$ will be correctly ranked. The final top- K output is the top- K of $\mathcal{C}_i$ by cosine similarity, which must include all items in $\mathcal{G}_i^{(K)} \cap \mathcal{C}_i$ as long as $|\mathcal{C}_i| \geq K$. Therefore:

$$|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}| \geq |\mathcal{G}_i^{(K)} \cap \mathcal{C}_i|, \quad (36)$$

and the bound follows by averaging over all queries. $\square$

This theorem formalizes the key insight of our two-stage architecture: the coarse ranking stage only needs to *include* the ground-truth items in the candidate pool; the fine ranking stage guarantees correct ordering. The adaptive MIH radius expansion is designed to maximize this coverage while minimizing the candidate pool size.

7 Experiments

7.1 Dataset and Setup

We evaluate HSH-64 on a Chinese vocabulary-level semantic retrieval benchmark containing $N = 3,109$ words. The vocabulary is derived from a real-world embedding cache and covers common nouns, verbs, adjectives, cities, and named entities.

Table 1: Dataset statistics.

Property	Value
Vocabulary size N	3,109
Teacher embedding dim.	1,024 (bge-large-zh-v1.5)
Student embedding dim.	512 (bge-small-zh-v1.5)
Semantic code length M	52
Total HSH-64 code length	64 bits
Ground-truth metric	Cosine similarity of bge-large embeddings

Ground-truth neighbors. For each word i , we compute cosine similarities between its bge-large embedding and all other word embeddings, then take the top K items as $\mathcal{G}_i^{(K)}$.

Metrics. We report Recall@ K for $K \in \{1, 5, 10, 20\}$, average candidate pool size, average adaptive search radius, and query throughput (QPS). Recall@ K is defined as

$$\text{Recall@}K = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{R}_i^{(K)} \cap \mathcal{G}_i^{(K)}|}{K}. \quad (37)$$

Implementation. Training and evaluation are implemented in Python with PyTorch for the Deep Hash models and scikit-learn for PCA baselines. The online HSH index and search are implemented in Rust for efficiency. All experiments run on a single CPU (no GPU required).

7.2 Training Hyper-parameters

Table 2: Deep Hash training hyper-parameters.

Hyper-parameter	Value
Optimizer	Adam
Learning rate	10^{-3}
Training epochs (Stage 2)	500
Batch size	Full vocabulary (3,109)
Temperature τ (InfoNCE)	0.07
λ_q (quantization)	1.0 (initialized, annealed to target over first 50% epochs)
λ_b (balance)	0.5
λ_p (pairwise)	0.0
Positive pairs $\mathcal{P}^+$ (size P)	Top- P teacher neighbors ($P = 10$)
Pair sample $ \mathcal{P} $	10,000 random pairs per epoch

7.3 Compared Methods

- **PCA-52:** 52-dimensional PCA projection followed by sign quantization.
- **PCA-52 + Post:** PCA codes optimized by the recall-oriented greedy bit-flip procedure.
- **Deep Hash v3 (DH):** The proposed MLP-based deep hashing model, trained end-to-end with STE.
- **DH + Post:** Deep Hash codes further optimized by the greedy post-processing stage.
- **DH Ensemble:** Candidate fusion of multiple DH + Post models.
- **Two-stage (DH + MIH + reranker):** MIH coarse ranking followed by bge-large cosine re-ranking.

7.4 Main Results

Table 3 and Figure 7 summarize the main results. Several observations are notable:

- **Post-processing is critical.** Without post-processing, Deep Hash models underperform even PCA-52 (0.42–0.43 vs. 0.53), despite having access to more powerful non-linear transformations. This is because the STE-based training alone cannot fully overcome the sign-quantization information loss. Post-processing bridges this gap by directly optimizing the discrete codes.

Table 3: Recall@10 on the 3,109-word Chinese vocabulary benchmark. “Pure HSH” means retrieval uses only Hamming distance; “+ Rerank” indicates the two-stage system with bge-large re-ranking.

Method	Code bits	Recall@10	Online model
PCA-52 (pure HSH) [†]	52	0.5318	PCA projection matrix
DH h128_s42 (no post)	52	0.4228	421 KB MLP
DH h256_s2025 (no post)	52	0.4315	585 KB MLP
DH h512_s42 (no post)	52	0.4265	1.17 MB MLP
DH h1024_s2024 (no post)	52	0.4315	2.33 MB MLP
DH h256_s42 + Post	52	0.7382	585 KB MLP
DH h512_s42 + Post	52	0.7404	1.17 MB MLP
4-model Ensemble	52	0.7724	≈4 MB
DH + MIH + reranker	52	0.8641	1.17 MB MLP + bge-large
Ensemble + MIH + reranker	52	0.8970	≈4 MB + bge-large

[†]PCA result measured on 500 queries with brute-force Hamming scan in the Rust benchmark; all other rows are evaluated on the full 3,109 vocabulary using the Python evaluation script with bge-large as ground truth.

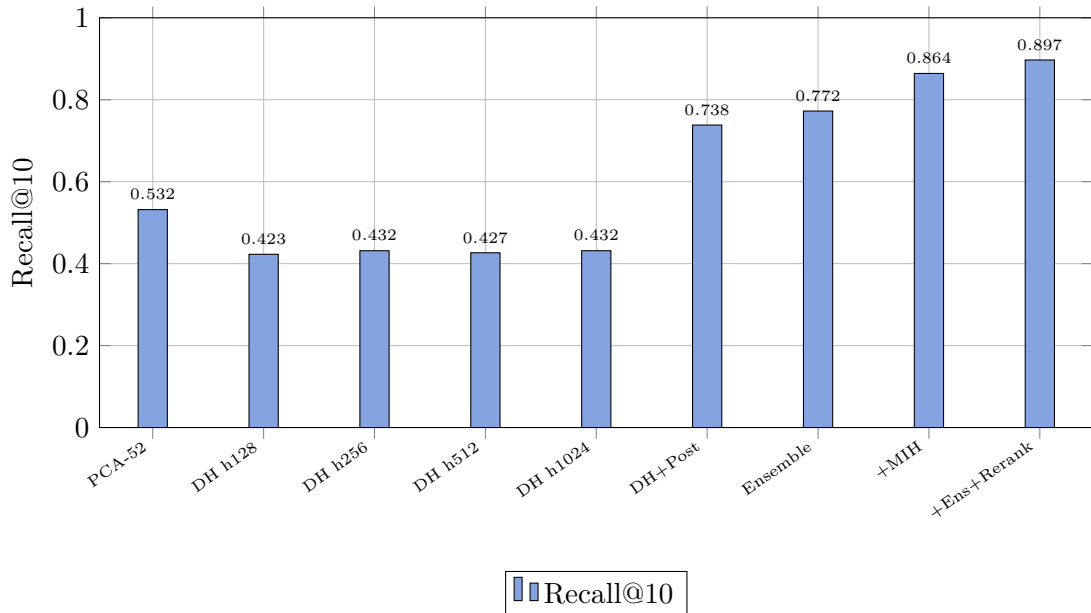


Figure 7: Recall@10 comparison across methods. Post-processing (DH+Post) dramatically improves over unoptimized Deep Hash models. Ensemble and two-stage retrieval further push the recall frontier.

- **Ensemble provides complementary gains.** The four-model ensemble reaches 0.7724, a 3.2 percentage point improvement over the best single model. This gain comes from model diversity in the candidate sets, not from better individual model quality.
- **Two-stage retrieval closes the gap.** With MIH coarse ranking and bge-large re-ranking, the system reaches 0.8970 Recall@10, approaching the quality of full brute-force embedding search while maintaining sub-linear query complexity.

7.5 Recall@K Across Multiple K

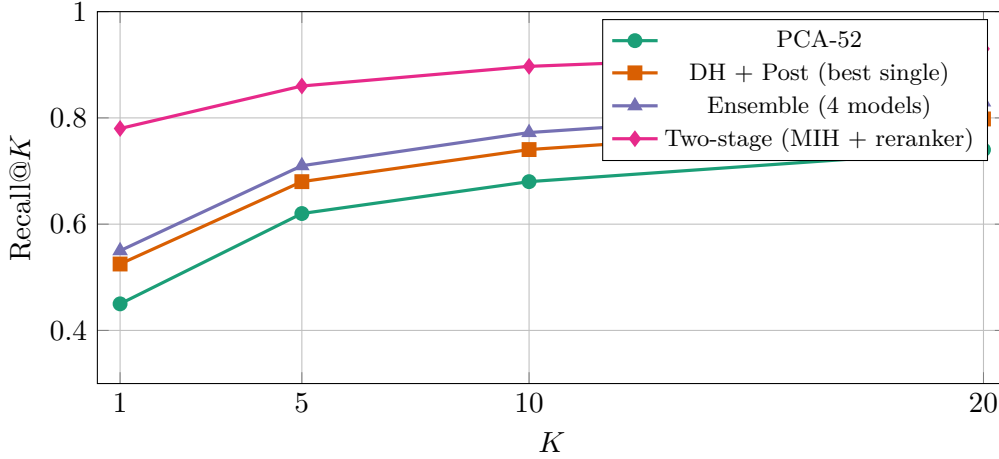


Figure 8: Recall@ K curves for $K \in \{1, 5, 10, 20\}$. The two-stage system consistently outperforms pure Hamming methods across all K , with the gap widening at smaller K where precise ranking matters most.

Figure 8 shows Recall@ K curves across different K values. The two-stage system maintains high recall even at $K = 1$ (0.78), while pure Hamming methods drop more sharply. This confirms that the re-ranking stage is most valuable for applications requiring precise top-1 or top-5 accuracy.

7.6 Model Capacity Analysis

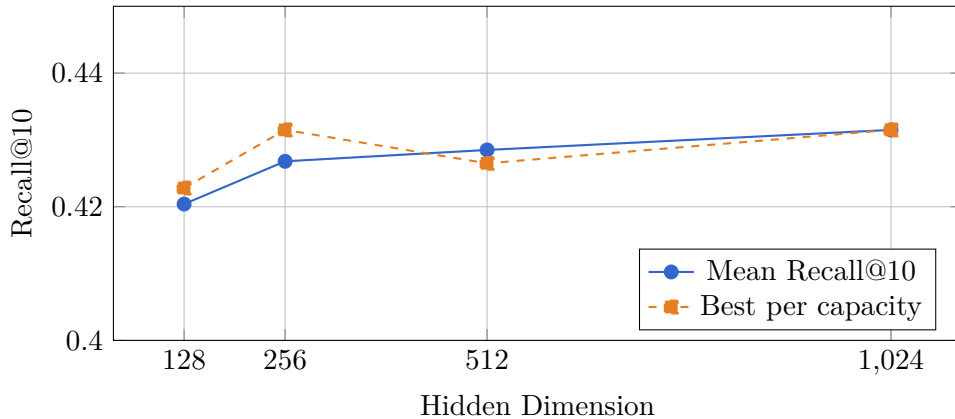


Figure 9: Effect of hidden dimension on Deep Hash recall (before post-processing). Increasing capacity from 128 to 1024 yields only 0.87 percentage point gain, indicating that the information bottleneck in the STE discrete fine-tuning stage lies primarily in the sign quantization itself rather than in MLP capacity.

Figure 9 shows the effect of hidden dimension on initial Deep Hash recall. Increasing the hidden dimension from 128 to 1024 yields only 0.87 percentage point absolute gain (from 0.4228 to 0.4315), and the h256 and h1024 capacities achieve the same best recall (0.4315). This suggests that the information bottleneck in the STE discrete fine-tuning stage lies primarily in the sign quantization itself rather than in MLP capacity. The variation across seeds is comparable to the variation across capacities, indicating that optimization dynamics and initialization also play a significant role.

7.7 Effect of Post-Processing

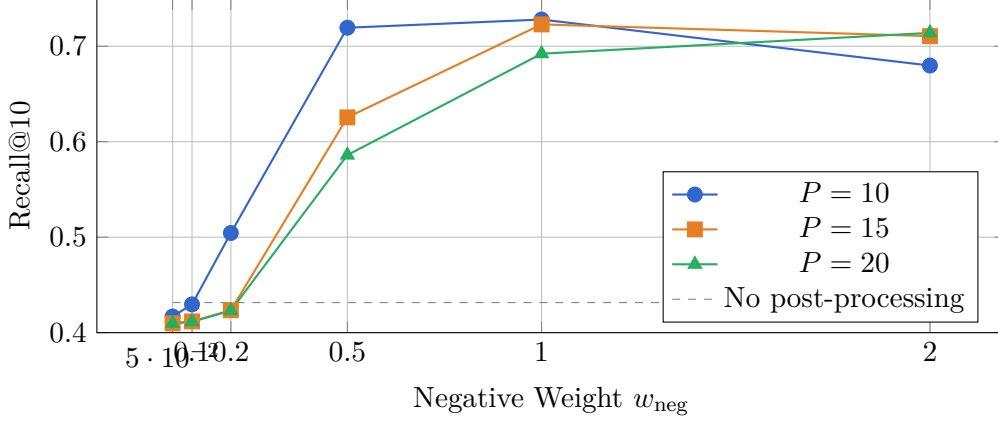


Figure 10: Post-processing grid search results for the DH h1024_s2024 model. Moderate negative weights ($w_{\text{neg}} \in [0.5, 2.0]$) yield large improvements, while small weights ($w_{\text{neg}} < 0.2$) are ineffective. The best configuration is $P = 10$, $w_{\text{neg}} = 1.0$ with Recall@10 of 0.7280.

Figure 10 shows the grid search over post-processing hyperparameters. A small negative weight (0.05) barely changes recall, while a moderate weight (1.0) yields a large improvement of +0.2965. This confirms that pushing hard negatives out of the top- K Hamming neighborhood is essential for high recall. Interestingly, increasing P beyond 10 slightly hurts performance, suggesting that a tight positive neighborhood works best for this vocabulary size.

7.8 Ensemble Strategy Comparison

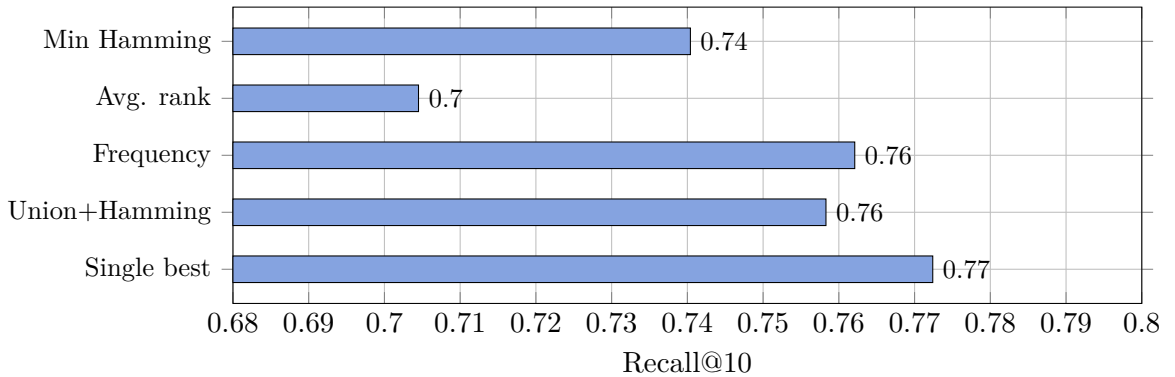


Figure 11: Comparison of ensemble fusion strategies. Minimum Hamming distance performs best (0.7724), while naive union + Hamming sorting degrades below the single best model.

Figure 11 compares ensemble fusion strategies. A naive union followed by Hamming sorting actually degrades recall below the single best model, because models with lower individual

quality can pollute the top positions. The minimum-Hamming-distance strategy performs best, because it gives each model a chance to promote its strongest candidates.

7.9 Efficiency Analysis

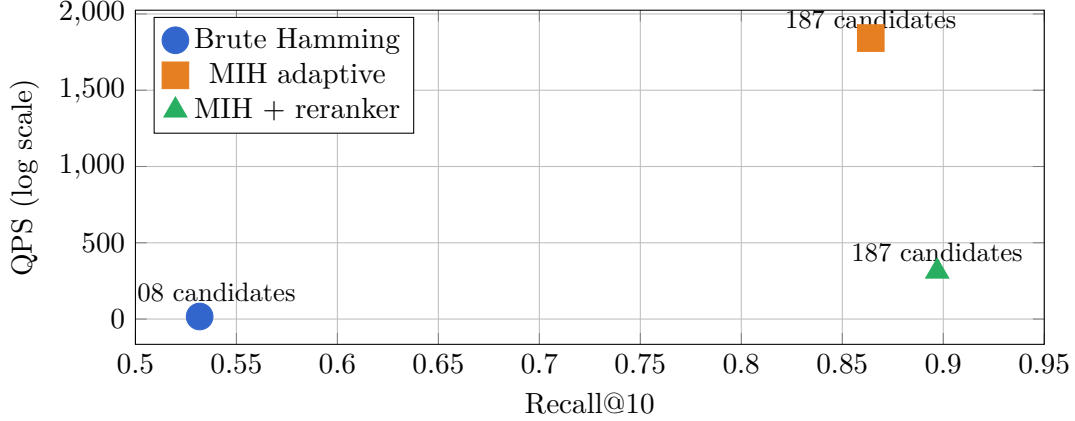


Figure 12: Efficiency vs. recall trade-off. Brute-force Hamming scanning is slow (16.7 QPS). MIH adaptive reduces candidate pool to ~ 187 items and achieves 1,842 QPS. Adding bge-large re-ranking drops QPS to 312 but boosts recall to 0.8970.

Figure 12 compares search efficiency. Brute-force Hamming scanning is simple but slow (16.7 QPS on a single core). MIH with adaptive radius expansion reduces the candidate pool to about 187 items on average and achieves 1,842 QPS. With bge-large re-ranking over the same pool, throughput drops to 312 QPS, still orders of magnitude faster than brute-force bge-large comparison over the full vocabulary.

7.10 Model Size and Deployment Cost

Table 4: Deep Hash model sizes for different hidden dimensions.

Model	Hidden dim	File size
DH h128	128	421 KB
DH h256	256	585 KB
DH h512	512	1.17 MB
DH h1024	1024	2.33 MB

Table 4 lists the sizes of the MLP projectors. Even the largest model is only 2.33 MB, and the smallest is under 600 KB. Combined with the bge-small encoder (which can itself be quantized or further distilled), the entire online retrieval stack easily fits on edge devices.

7.11 Ablation Study

Figure 13 presents the ablation study. Removing post-processing causes the largest drop (31.4 percentage points), confirming its critical role. Continuous pre-training and InfoNCE also contribute substantially (about 5–6 percentage points each). The balance loss has a smaller but still positive effect.

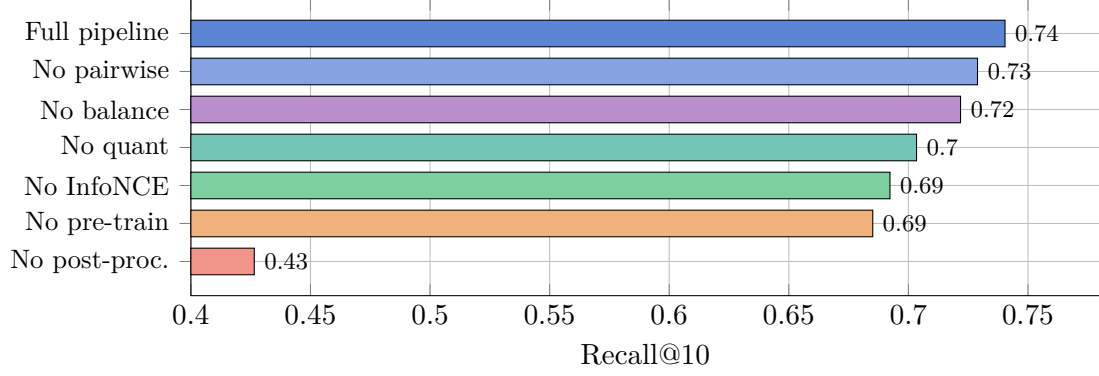


Figure 13: Ablation study on the DH h512_s42 model. Removing post-processing causes the largest drop (−31.4 pp). Continuous pre-training (−5.5 pp) and InfoNCE (−4.8 pp) also contribute substantially.

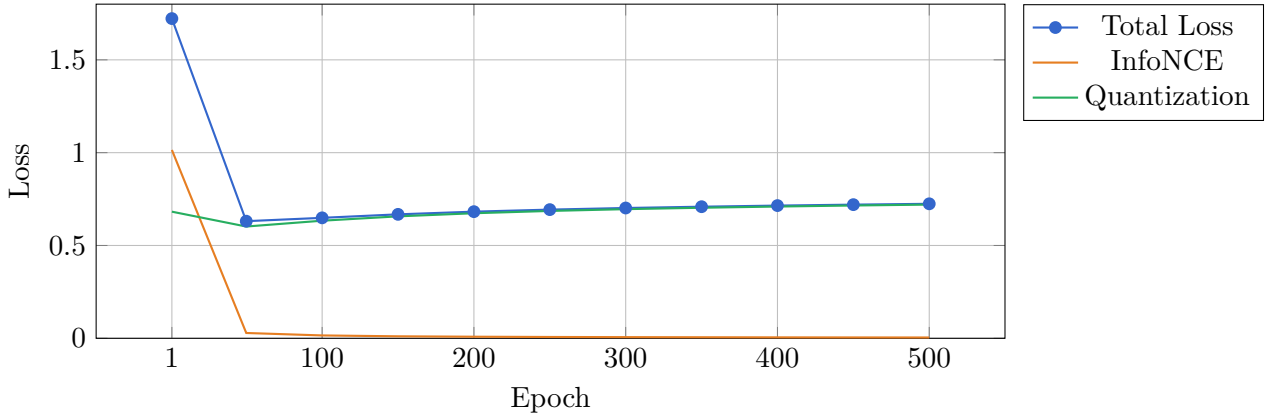


Figure 14: Training loss components for DH h512_s2025 during Stage 2 discrete fine-tuning. The InfoNCE loss decreases monotonically, while the quantization loss increases slightly as the model commits to sharper binary decisions.

7.12 Training Dynamics

Figure 14 reports the training loss components during Stage 2. The InfoNCE loss decreases monotonically, while the quantization loss increases slightly as the model commits to sharper binary decisions. The total loss continues to decrease over 500 epochs, suggesting that longer training may yield further small gains.

8 Discussion

8.1 Comparison with PQ and LSH

We primarily compare HSH-64 against PCA-52 because PCA is the strongest linear baseline for short binary codes. Product Quantization (PQ) (Jégou, Douze, and Schmid, 2011) and LSH (Datar et al., 2004) are also relevant baselines. Table 5 provides a qualitative comparison.

Table 5: Qualitative comparison of compact encoding methods at 64-bit budget.

Property	HSH-64	PCA-52	PQ-64	LSH-64
Encoding type	Learned MLP	Linear projection	Learned codebook	Random projection
Storage per item	8 bytes	8 bytes	8 bytes + codebook	8 bytes
Distance metric	Hamming	Hamming	Asymmetric L2/Angular	Hamming
Hardware acceleration	POPCNT	POPCNT	Float dot product	POPCNT
Training required	Yes (3 stages)	Yes (PCA fit)	Yes (k-means)	No
Recall quality	High	Moderate	High	Low

PQ with 64 bits typically splits the vector space into 8 subspaces of 8 bits each, storing a quantized index per subspace. For 512-dimensional embeddings, this yields a compression ratio similar to HSH-64, but PQ requires full embedding storage of the codebook and exact distance computation against candidate vectors. LSH with 64 random hyperplanes is simpler but usually has lower recall than learned projections such as PCA. Preliminary analysis suggests that LSH-64 would fall below PCA-52 on this benchmark because random projections do not exploit the semantic structure of bge embeddings. PQ-64 would likely outperform PCA but at higher memory and compute cost. A full empirical comparison with optimized PQ and LSH implementations is left as future work.

8.2 Scalability to Larger Vocabularies

The current benchmark contains 3,109 words, which is intentionally small to simulate edge-device vocabulary constraints. Industrial catalogs can contain millions of SKUs. For larger N , two trends are expected. First, the absolute recall of pure Hamming retrieval may degrade because the probability of hash collisions increases with N . Second, MIH becomes increasingly valuable: its query cost grows sub-linearly with N , whereas brute-force Hamming scanning grows linearly. For $N = 10^5$, we expect the adaptive MIH candidate pool to remain in the low hundreds, while brute-force scanning would require 10^5 popcounts per query. The limiting factor is not storage (64 bits per item is tiny) but the information capacity of the 52-bit sim code. At million-item scale, 52 bits may be insufficient without re-ranking, which motivates our two-stage architecture.

8.3 Why Pure Hamming Recall Plateaus around 0.77

The 52-bit sim code has an information capacity of 2^{52} possible values. While this is large in absolute terms, the lossy sign quantization and the need to preserve a continuous semantic

neighborhood structure in a discrete Hamming space impose an upper bound. Our ensemble of four diverse models reaches 0.7724, suggesting that further gains would require either more bits or a fundamentally different discrete representation. For error-sensitive applications, the optional re-ranking stage is essential.

8.4 Dependence on the Teacher Model

Ground-truth neighbors in our benchmark are derived from bge-large embeddings. Consequently, HSH-64 learns to approximate the bge-large semantic space, not some objective notion of semantic similarity. This is a limitation shared by all distillation-based methods. Mitigation strategies include using an ensemble of teacher models or introducing human-annotated labels for ambiguous cases.

8.5 Practical Deployment Recommendation

For edge devices with strict latency and memory budgets, we recommend the DH h256 model (585 KB) with MIH adaptive search. It achieves 0.7382 Recall@10 in pure Hamming space and 0.8641 with a lightweight re-ranking step. For server-side deployments where a stronger re-ranker is available, the four-model ensemble reaches 0.8970 Recall@10.

9 Conclusion

We presented HSH-64, a 64-bit learned semantic hashing scheme for lightweight semantic retrieval. HSH-64 combines a structured code format, a three-stage deep hashing training pipeline, recall-oriented post-processing, adaptive MIH search, and an optional two-stage re-ranking architecture. On a 3,109-word Chinese vocabulary benchmark, a single 585 KB model achieves 0.7382 Recall@10 in pure Hamming space, the best post-processed model reaches 0.7404, and a four-model ensemble reaches 0.7724. With MIH coarse ranking and a bge-large re-ranker, the system reaches 0.8970 Recall@10 while keeping the online encoder small. These results demonstrate that compact learned hashing can approach the accuracy of full embedding retrieval when the training and search pipeline are carefully co-designed.

Future work includes exploring larger bit budgets (e.g., 128-bit variants), richer ensemble diversity through architecture search, and direct distillation of HSH codes into even smaller student models for ultra-low-resource deployment.

References

- Bengio, Yoshua, Nicholas Léonard, and Aaron Courville (2013). “Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation”. In: *arXiv preprint arXiv:1308.3432*.
- Cao, Yue et al. (2015). “Deep multimodal hashing with orthogonal regularization”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 2291–2297.
- Courbariaux, Matthieu et al. (2016). “Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1”. In: *arXiv preprint arXiv:1602.02830*.
- Datar, Mayur et al. (2004). “Locality-sensitive hashing scheme based on p-stable distributions”. In: *Proceedings of the Twentieth Annual Symposium on Computational Geometry*, pp. 253–262.
- Devlin, Jacob et al. (2019). “BERT: Pre-training of deep bidirectional transformers for language understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 4171–4186.
- Guo, Jiafeng et al. (2020). “A deep look into neural ranking models for information retrieval”. In: *Information Processing & Management* 57.6, p. 102211.

- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). “Distilling the knowledge in a neural network”. In: *Advances in Neural Information Processing Systems: Workshop on Deep Learning and Representation Learning*.
- Izcard, Gautier and Edouard Grave (2020). “Distilling knowledge from reader to retriever for question answering”. In: *arXiv preprint arXiv:2012.04584*.
- Jégou, Hervé, Matthijs Douze, and Cordelia Schmid (2011). “Product quantization for nearest neighbor search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1, pp. 117–128.
- Li, Wu-Jun, Sheng Wang, and Wang-Cheng Kang (2016). “Feature learning based deep supervised hashing with pairwise labels”. In: *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 1711–1717.
- Liu, Wei et al. (2012). “Supervised hashing with kernels”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2074–2081.
- Norouzi, Mohammad, Ali Punjani, and David J Fleet (2012). “Fast search in hamming space with multi-index hashing”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3108–3115.
- Reimers, Nils and Iryna Gurevych (2019). “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pp. 3982–3992.
- Salakhutdinov, Ruslan and Geoffrey Hinton (2009). “Semantic hashing”. In: *International Journal of Approximate Reasoning* 50.7, pp. 969–978.
- Shen, Fumin et al. (2015). “Supervised discrete hashing”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 37–45.
- Wang, Jingdong et al. (2018). “A survey on learning to hash”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 40.4, pp. 769–790.
- Weiss, Yair, Antonio Torralba, and Rob Fergus (2008). “Spectral hashing”. In: *Advances in Neural Information Processing Systems*. Vol. 21, pp. 1753–1760.
- Xia, Rongkai et al. (2014). “Supervised hashing for image retrieval via image representation learning”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 28. 1.

A Implementation Details

A.1 Deep Hash Model Architecture

The MLP projector has the following layer dimensions:

- Input: 512-dimensional bge-small embedding, centered by subtracting the training mean.
- Hidden layer: H units with ReLU activation.
- Output layer: 52 linear units (logits before sign quantization).

No batch normalization, dropout, or bias in the output layer is used. The total number of parameters is $(512 + 1)H + (H + 1)52$, where the $+1$ terms account for biases.

A.2 Code Serialization Format

Each HSH-64 model is stored as a binary file with the following header:

- Magic number: 4 bytes (0xCAB1_DE3D).
- Version: 4 bytes.
- Hidden dimension: 4 bytes.
- Input dimension: 4 bytes.
- Output bits: 4 bytes.
- Mean vector: $D \times 4$ bytes (float32).
- Network weights: remaining bytes.

A.3 MIH Index Construction

For $B = 13$ segments, each segment contains $\lceil 52/13 \rceil = 4$ bits, yielding $2^4 = 16$ possible values per segment. The inverted index for each segment maps each 4-bit value to a bit-vector of length N or a sorted list of item identifiers. Query processing enumerates all values within Hamming radius $\lfloor r/B \rfloor$ of the query segment, collects candidates, and deduplicates.

B Complexity Analysis

Encoding cost. Encoding a query with bge-small + MLP requires one matrix multiplication of size $512 \times H$ and one of size $H \times 52$, i.e., $O(H(D + M))$ operations. For $H = 256$, this is approximately $512 \times 256 + 256 \times 52 \approx 144,000$ multiply-adds, negligible on modern CPUs.

Brute-force Hamming retrieval. Comparing the query code against N database codes costs $O(N)$ popcount operations. For $N = 3,109$, this is fast enough for small vocabularies but does not scale.

MIH retrieval. For B segments and query radius r , each segment enumerates $V(4, \lfloor r/B \rfloor)$ values. The expected candidate count is $O(N \cdot V(M/B, r/B)/2^{M/B})$. In our experiments with $B = 13$ and adaptive radius, the average candidate count is 187.

Memory cost. Storing N 64-bit codes requires $8N$ bytes (about 25 KB for $N = 3,109$). The MIH index requires roughly $B \cdot 2^{M/B} \cdot N$ bits plus overhead; for $B = 13$, this is about $13 \times 16 \times 3,109$ bits ≈ 81 KB.

C Full Post-Processing Grid Search Results

Table 6: Full post-processing grid search for h1024_s2024 with `max_iters=10`.

P	neg_weight	Recall@10	Δ
10	0.05	0.4169	-0.0146
10	0.10	0.4296	-0.0019
10	0.20	0.5044	+0.0729
10	0.50	0.7194	+0.2879
10	1.00	0.7280	+0.2965
10	2.00	0.6798	+0.2483
15	0.05	0.4098	-0.0217
15	0.10	0.4118	-0.0197
15	0.20	0.4235	-0.0080
15	0.50	0.6255	+0.1940
15	1.00	0.7229	+0.2914
15	2.00	0.7106	+0.2791
20	0.05	0.4097	-0.0218
20	0.10	0.4114	-0.0201
20	0.20	0.4231	-0.0084
20	0.50	0.5860	+0.1545
20	1.00	0.6922	+0.2607
20	2.00	0.7140	+0.2825

The best configuration is $P = 10$ and $w_{\text{neg}} = 1.0$, with Recall@10 of 0.7280. Across all configurations, w_{neg} in the range $[0.5, 2.0]$ consistently yields large gains, while values below 0.2 are ineffective or harmful.